



Nebula KnowLabTM

TAKE-HOME ASSIGNMENT

Supermarket Ops Agent

Nebula KnowLab · Engineering

**Run an entire Indian kirana store from a chat window —
with an agent, not a menu.**

Interface: Telegram only

Harness: your choice (agent SDK)

Duration: 5 Calendar Days

1. The brief

Build a conversational agent that runs a small Indian supermarket / kirana store end-to-end. The owner operates the whole shop from **Telegram, in plain language** — receiving stock, cutting bills, checking what's left, running customer credit, closing the day, and pulling **invoices and analysis decks** on demand. There is **no web app, no admin panel, no forms**. The chat is the product.

This is the whole point. It is **not** a CRUD app with a chatbot in front of it. It's an **agent** that reasons over messy human requests and keeps a store's books consistent. We don't hand you a tool list — **you author the skills and tools** that run the store and let the model orchestrate them. If you find yourself writing a large **if/elif** intent router full of regexes, you've taken the wrong turn.

Build on **any modern agent harness** — your choice. Designing the skill and tool surface, and picking a harness you can justify, is the core of the exercise.

Harness options: Claude Agent SDK · deep agent · Vercel AI SDK (or equivalent).

2. The domain – Indian kirana / supermarket

Model a real Indian grocery store. This is where the “did they actually think about it” signal lives.

- **Currency:** ₹ (INR).
- **Products:** real SKUs — Aashirvaad Atta 5kg, Tata Salt 1kg, Amul Butter 100g, Fortune Sunflower Oil 1L, Maggi 70g, Parle-G, Surf Excel — plus loose items (sugar, rice, dal by the kg).
- **Units:** kg / g / litre / ml / packet / dozen / piece. Loose vs packaged.
- **GST:** compute it correctly. Many staples are **0%** (loose atta/rice/fresh produce), packaged staples **5%**, FMCG like chocolates/soaps **12–18%**. Intra-state = **CGST + SGST** split; each item carries an **HSN** code and a tax slab. A bill must show the tax breakup and round correctly.
- **Payments:** Cash / UPI / Card — record the mode and a reference; no real gateway.
- **Khata (credit ledger):** customers buy on credit and settle later — a first-class kirana concept. “Put ₹500 on Ramesh's credit”, “What's Ramesh's balance?”, “Ramesh paid ₹300”.
- **Stock discipline:** every SKU has cost price, MRP/sell price, quantity and a reorder level. Selling decrements stock atomically.

The owner types in plain, terse English — real-shopkeeper phrasing.

3. What the owner must be able to do

These are *capabilities*, not fixed commands — the agent decides which tools to call.

Intent	Example message
Receive stock	50 packets of Maggi came in, cost ₹12, MRP ₹14
Add a new product	new item: Amul Butter 100g, GST 12%, MRP ₹62
Cut a bill	make a bill: 2kg sugar, 1 Aashirvaad atta 5kg, 4 Maggi, 1 Amul butter, UPI
Edit a bill mid-build	drop the butter, make it 6 Maggi
Stock query	how much sugar is left?
Low-stock / reorder	what's running out?
Credit (khata)	put ₹500 on Ramesh's credit · Ramesh paid ₹300 · Ramesh's balance?
Daily close	today's sales? / close the day → total, tax collected, cash vs UPI, top items
Invoice as PDF	send me that bill as a PDF → a clean, GST-correct invoice document
Analysis deck	make this week's sales analysis deck → a PPTX with charts & insights
Set a preference	always assume UPI unless I say cash · default atta = Aashirvaad 5kg → remembered across chats

When a request is genuinely ambiguous (e.g. add **atta** → “Which one — Aashirvaad 5kg or loose?”), the agent should **ask a clarifying question** rather than guess. That clarification must come from the **model**, not a hardcoded branch.

4. The hard parts – this is the actual test

A working demo is easy. These are what separate a real agent from a toy. Handle them, and describe *how* in your README.

1. **Grounding.** Prices, GST slabs and stock come from the DB via tools. Never invent a product or a price.
2. **Oversell guard.** Stock can't go negative. Billing 10 when 6 are in stock is refused at the *tool* layer, not the prompt.
3. **GST correctness.** Per-item slab, CGST/SGST split, rounding, and a legible tax breakup on the bill.
4. **Multi-turn bills.** A bill builds over several messages, supports edits, and only decrements stock on *finalize*.
5. **Idempotency.** Telegram redelivers updates. A retried “finalize” must not double-bill or double-decrement stock.
6. **Concurrency.** Two bills — or a sale plus a stock-in — in flight at once must not corrupt stock.
7. **Guardrails.** Don't sell below cost, don't delete stock, don't settle a khata that doesn't exist — confirm or refuse.
8. **Real artifacts.** A proper GST invoice (PDF) and a business-analysis deck (PPTX) with real charts — generated by the agent's own tools, not screenshots or plain text.
9. **Memory across sessions.** Standing preferences (default payment, preferred brand, shop name/GSTIN on invoices) persist across chats — start a */new* chat and they still apply. Memory lives outside the context window, not just in the conversation.

5. Architecture requirements

- **You design the skills and tools.** We deliberately don't give you a tool list. Deciding what the skills are (inventory, billing, credit, analytics, document generation...), how thin each tool should be, and how the agent composes them is *the* thing we're evaluating. Author them the way your harness expects — skill files, tool schemas, subagents.
- **Agent-first — the model orchestrates.** Natural language goes through the model + your tools. A regex/keyword router doing the real work is an automatic fail.
- **Business rules live in the skills/tools, not the prompt.** Oversell guard, GST maths, idempotency and khata rules are enforced where the data changes — not hoped for in the system prompt.
- **A real control loop.** Observe → reason → act (call tool) → feed result back → continue, chaining multiple tool calls within one turn as needed.

- **Persistence & memory.** SQLite or Postgres. Survives a restart. Stock, khata, bills *and owner preferences* are durable — a */new* chat clears the conversation but not what the store knows about how this owner works.

On frameworks. Use any modern agent harness — **Claude Agent SDK, deep agent, Vercel AI SDK**, or an equivalent. We are explicitly **not** asking for a LangGraph-style, node-per-command state machine — that misreads the task.

6. Deliverables

- **A live bot we can message** — deploy it and put the Telegram **bot handle (@username)** in your README, kept running while we review, so we can drive it through the scenarios in §3 ourselves.
- **Built on a modern agent harness** — Claude Agent SDK, deep agent, Vercel AI SDK, or equivalent.
- **Your skills & tools** — the capability surface you designed to run the store. This is the heart of what we grade.
- **PDF invoices** — any bill can be produced as a clean, GST-correct PDF invoice, generated by the agent.
- **PPTX analysis deck** — on request, the agent generates a PowerPoint analysing the store (sales, top items, stock health, GST collected) with real charts.
- **README (~1 page)** — the harness you picked and **why**; how your control loop works; your skill/tool design; and how you solved each hard part.
- **A 4–5 min recording** — receive stock → multi-item bill with an edit → oversell guard → khata cycle → generate a PDF invoice → generate the analysis deck → set a preference, start a */new* chat, show it's remembered.
- **A private GitHub repository** — invite the following collaborators: **Aswath363, akshaiP, ashwanthnebula**, with a clean commit history showing progression.

7. Stretch – optional, to stand out

- Branded / templated invoice PDFs
- Scheduled weekly analysis deck, auto-sent
- Reorder suggestions from sales velocity
- Expiry / batch tracking with FEFO
- Voice-note orders (transcribe → bill)

- Multi-language (Hindi / Tamil)
- Barcode / product photo → identify item
- Khata payment reminders

Submission

Submit within **5 calendar days** of receiving this task.

Ping us with questions – knowing *what* to ask is part of the signal.

Good luck — we're excited to see what you build. · Nebula KnowLab, Engineering